

OBJECT ORIENTED PROGRAMMING

Write a C++ program to print pascal triangles.

Program 1: Pascal Triangle

```
#include <iostream>
using namespace std;

int fact(int n);

int nCr(int n, int r) {
    return fact(n) / (fact(r) * fact(n - r));
}

int main() {
    int rows;
    cout << "Enter number of rows: ";
    cin >> rows;

    for (int i = 0; i < rows; i++) {
        // Print spaces
        for (int j = 0; j < rows - i - 1; j++) {
            cout << " ";
        }
        // Print Pascal's Triangle values
        for (int j = 0; j <= i; j++) {
            cout << nCr(i, j) << " ";
        }
        cout << endl;
    }
    return 0;
}

int fact(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    return fact;
}
```

Find out the sum of even numbers between 1 to 50 using loops. (Also did Sum of Odd

Numbers between 1 to 50)

Program 2: Sum of even and odd numbers

```
#include <iostream>
```

```
using namespace std;
```

```
class Numbersum{
```

```
private:
```

```
    int start, end;
```

```
    int oddsum = 0;
```

```
    int evensum = 0;
```

```
public:
```

```
    void Range();
```

```
    void sum();
```

```
    int geteven_sum();
```

```
    int getodd_sum();
```

```
};
```

```
void Numbersum::Range() {
```

```
    cout << "Enter the starting number: ";
```

```
    cin >> start;
```

```
    cout << "Enter the ending number: ";
```

```
    cin >> end;
```

```
}
```

```
void Numbersum::sum() {
```

```
    for (int i = start; i <= end; i++) {
```

```
        if (i % 2 == 0)
```

```
            evensum += i;
```

```
        else
```

```
            oddsum += i;
```

```
    }
```

```
}
```

```
int Numbersum::geteven_sum() {
```

```
    return evensum;
```

```
}
```

```
int Numbersum::getodd_sum() {  
    return oddsum;  
}  
  
void display(Numbersum N) {  
    cout << "Even sum of numbers: " << N.geteven_sum() << endl;  
    cout << "Odd sum of numbers : " << N.getodd_sum() << endl;  
}  
  
int main() {  
    Numbersum N;  
    N.Range();  
    N.sum();  
    display(N);  
  
    return 0;  
}
```

Write a program in C++ to sort the numbers in an array using separate functions for read, display, sort and swap. The objective of this assignment is to learn the concepts of input, output, functions, call by reference in C++.

Program 3: Sorting Number in an array

```
#include <iostream>
using namespace std;

class Sorting {
private:
    int arr[100], n;

public:
    void Read();
    void Display();
    void swap(int j);
    void sorting();
};

void Sorting::Read() {
    cout << "Enter the number of elements: ";
    cin >> n;
    cout << "Enter the elements of the array:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}

void Sorting::Display() {
    // cout << "The Final Array Is:\n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << "\t";
    }
    cout << endl;
}

void Sorting::swap(int j) {
    int temp;
    temp = arr[j];
    arr[j] = arr[j + 1];
    arr[j + 1] = temp;
}
```

```
}
```

```
void Sorting::sorting() {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(j);  
            }  
        }  
    }  
}
```

```
int main() {  
    Sorting S;  
    S.Read();  
    cout << "\nArray before sorting:\n";  
    S.Display();  
    S.sorting();  
    cout << "\nArray after sorting:\n";  
    S.Display();  
    return 0;  
}
```

Write a program in C++ to perform following operations on complex numbers Add, Subtract, Multiply, Divide, Complex conjugate. Design the class for complex number representation and the operations to be performed. The objective of this assignment is to learn the concepts of classes and objects.

Program 4: Performing Operations On Two Complex Numbers

```
#include<iostream>
#include<math.h>

using namespace std;

class Complex
{
private:
    int real, img;

public:
    void display();
    void read();
    void add(Complex);
    void subtract(Complex);
    void multiply(Complex);
};

void Complex::read(){
    cout << "Enter real part: ";
    cin >> real;
    cout << "Enter imaginary part: ";
    cin >> img;
}

void Complex::display(){
    cout << real << " + " << img << " i " << endl;
}

void Complex::add(Complex C1){
    Complex Sum;
    Sum.real = real + C1.real;
    Sum.img = img + C1.img;
```

```

    Sum.display();
}

void Complex::subtract(Complex C1){
    Complex Difference;
    Difference.real = real - C1.real;
    Difference.img = img - C1.img;
    Difference.display();
}

void Complex::multiply(Complex C1){
    Complex Product;
    Product.real = (real * C1.real) - (img * C1.img);
    Product.img = (real * C1.img) + (img * C1.real);
    Product.display();
}

int main(){
    Complex C1, C2;
    cout << "Enter first complex number:" << endl;
    C1.read();
    cout<<endl;
    cout << "Enter second complex number:" << endl;
    C2.read();
    cout<<endl;
    cout << "Addition: ";
    C2.add(C1);
    cout << "Subtraction: ";
    C2.subtract(C1);
    cout << "Multiplication: ";
    C2.multiply(C1);
    return 0;
}

```

Write a C++ program that illustrates the concept of Function over loading.

Program 5: Function Over Loading.

```
#include<iostream>
using namespace std;

class Operation{
    private:
        int a, b;

    public:
        int add(int a, int b);
        int add(int a, int b, int c);
        float add(float a, float b);
};

int Operation::add(int a, int b) {
    return a + b;
}
int Operation::add(int a, int b, int c) {
    return a + b + c;
}
float Operation::add(float a, float b) {
    return a + b;
}

int main(){
    Operation O;
    cout << "Addition of two numbers: " << O.add(10, 20) << endl;
    cout << "Addition of three numbers: " << O.add(10, 20, 30) << endl;
    cout << "Addition of float numbers: " << O.add(2.5f, 3.5f) << endl;
}
```